E-Graph-Augmented Tool-in-the-Loop Agent for High-Coverage RTL Verification

[Student 1 (cc5583)], [Student 2 (mz3143)], [Student 3 (yc4653)]
COMS 6998: AI-Native Computing | Fall 2026
Project Proposal | Due October 5, 2026, 11:59 PM ET

Track / topic: B: AI for Computing; Automated Hardware Verification

Format. Write **two pages**, excluding references.

I. Problem and Motivation

Problem. While Large Language Models (LLMs) show promise in hardware design, recent benchmarks like CVDP demonstrate that they severely struggle with generating functionally correct, high-coverage verification code (e.g., testbenches and assertions) [1]. Furthermore, relying purely on text-based compilation logs for iterative repair often traps LLM agents in syntactic loops, failing to grasp the underlying structural dependencies of the RTL.

Motivation. Verification accounts for over 50% of the chip design cycle. By transitioning from simplistic pass/fail text logs to dense, structural feedback mechanisms—such as Execution Coverage and E-graph (Equivalence Graph) representations—we can guide LLM agents to semantically understand logic paths, rapidly escape local minima during debugging, and achieve industrial-grade verification coverage with fewer costly simulator invocations.

Research question. Can integrating E-graph-based structural representations and coverage-guided simulator feedback into an iterative LLM Agent significantly improve verification coverage and Token Return-on-Investment (Token ROI) compared to traditional text-based tool-in-the-loop agents?

II. Related Work

Our approach bridges three emerging domains. First, the CVDP benchmark identifies the profound weakness of LLMs in verification tasks [1]. Second, the LLM4Cov framework illustrates that execution-aware, coverage-guided agentic learning significantly outperforms zero-shot generation [2]. Finally, recent advancements in Equality Saturation and E-graphs (e.g., SmoothE) demonstrate their capability to compactly represent functionally equivalent expressions, bypassing phase-ordering problems in logic synthesis [3]. We build on these by utilizing E-graphs not just for compilation, but as a semantic feedback mechanism for LLM agents.

III. Proposed Approach

We propose a multi-turn, E-graph-augmented agent framework designed to maximize testbench coverage efficiently. The pipeline consists of three core components: 1. **Initial Generation & Structural Parsing:** The LLM generates initial SystemVerilog stimuli based on the specification. An open-source E-graph tool (e.g., *egg*) is used to parse the target RTL into an equivalence graph, abstracting its semantic logic. 2. **Coverage-Guided Execution:** An EDA simulator (e.g., Verilator or Icarus Verilog) executes the generated code to produce dense metrics (line/toggle/branch coverage) rather than binary pass/fail signals. 3. **E-Graph Augmented Refinement:** When coverage is low or assertions fail, the agent queries the E-graph representation. Instead of blind text guessing, the E-graph reveals equivalent logical paths and uncovered structural bottlenecks. This semantic prompt guides the LLM to generate targeted mutations, effectively improving coverage while minimizing redundant simulator calls.

IV. Evaluation Plan

We will evaluate the proposed framework on the Design Verification and AutoEval-ECov subsets of CVDP [1], [2]. To isolate the contribution of semantic E-graph feedback, we compare three configurations: 1) **Text-only Agent**: receives only compilation errors, simulation logs, and pass/fail feedback.

2) **Coverage-Guided Agent**: additionally receives raw line, branch, and toggle coverage information, including uncovered RTL predicates.

3) **E-Graph Semantic Coverage Agent**: receives the same execution coverage feedback as (2), but uncovered predicates are normalized and grouped into semantic equivalence classes using an E-graph before being provided to the LLM.

We will measure final branch/toggle coverage, Pass@1, number of generated tests, number of simulator invocations, token consumption, and Token ROI [4]. The comparison between (1) and (2) quantifies the value of execution-aware feedback, while the comparison between (2) and (3) isolates the contribution of E-graph-based semantic representation.

V. Timeline and Team Responsibilities

Date	Planned work and deliverables
Oct 19	Set up the open-source EDA simulator and the CVDP-ECov dataset.
Nov 2	Implement the baseline text-only agent and establish initial coverage and Token ROI metrics.
Nov 23	Integrate the *egg* library to extract E-graph representations from RTL designs.
Dec 7	Complete full evaluation of the E-graph augmented agent and collect comparison metrics.
Dec 18	Finalize poster presentation and project report.

Presentations. Midterm presentation: Nov 6. Final poster session: Dec 11.

Team roles. *[Student 1]*: EDA simulator integration and Coverage-guided feedback loop setup. *[Student 2]*: E-graph (*egg*) engine integration and semantic prompt design. *[Student 3]*: LLM API orchestration, Token ROI evaluation, and report writing.

References

[1] N. Pinckney, Y.-D. Tsai, C. Deng, M. Liu, *et al.*, "Comprehensive Verilog Design Problems: A Next-Generation Benchmark Dataset for Evaluating Large Language Models and Agents on RTL Design and Verification," *arXiv preprint arXiv:2506.14074*, 2025.

[2] H. Zhang, Z. Yu, C.-T. Ho, H. Ren, *et al.*, "LLM4Cov: Execution-Aware Agentic Learning for High-coverage Testbench Generation," *arXiv preprint arXiv:2602.16953*, 2026.

[3] Y. Cai, K. Yang, C. Deng, C. Yu, and Z. Zhang, "SmoothE: Differentiable E-Graph Extraction," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025, pp. 1020–1034.

[4] J. Deng, Z. Chen, X. Wei, T. Xing, *et al.*, "Can AI Agents Really Complete RTL-to-GDS? Lessons from Benchmarking Tool-Interactive EDA Workflows," *arXiv preprint arXiv:2607.17528*, 2026.